Diplomado en Arquitectura de Software

Módulo 1 - Fundamentos de Ingeniería de Software

Prof. César Augusto Vega Fernández

Facultad de **Ingeniería**

VIGILADA MINEDUCACIÓN



Universidad de La Sabana

Universidad de La Sabana | Facultad de **Ingeniería**


Edificio Ad Portas

Bienvenidos

Bienvenidos al módulo de **Fundamentos de Ingeniería de Software**. En este curso, exploraremos los conceptos esenciales que todo ingeniero de software debe dominar para desarrollar sistemas robustos y eficientes. Desde la arquitectura de software hasta el modelado orientado a objetos y los atributos de calidad, este módulo sentará las bases necesarias para comprender la ingeniería de software en su totalidad.

Ingeniería de Software

La **Ingeniería de Software** es fundamental para el desarrollo de sistemas de calidad. Se basa en principios como la **gestión de proyectos**, el **análisis de requisitos** y la **calidad del software**. Esta disciplina busca optimizar el proceso de desarrollo y garantizar la fiabilidad y eficiencia de los sistemas. Es esencial para el éxito en el mundo tecnológico actual.



Principios fundamentales

Gestión de la calidad de software

Reutilización de componentes

Documentación detallada

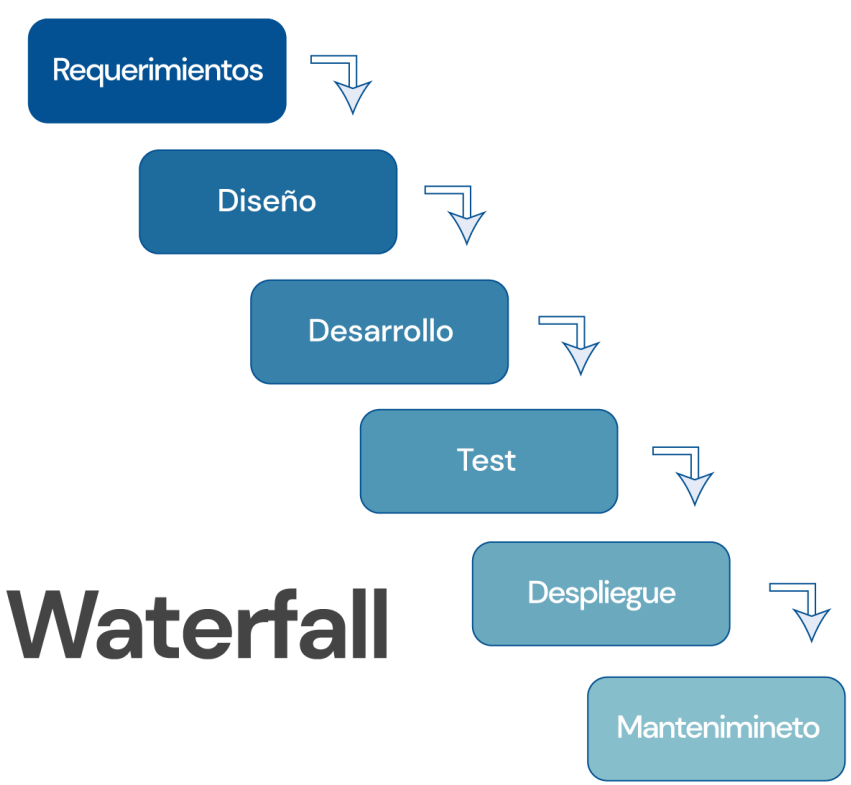
Colaboración interdisciplinaria



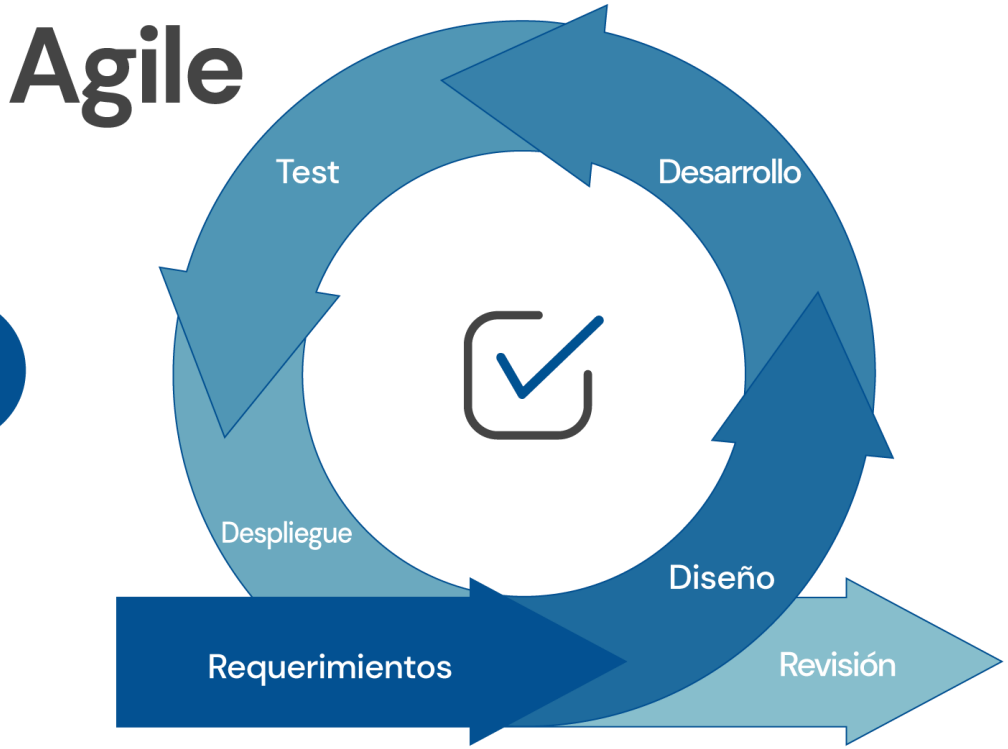
Lago con vista al Edificio Ad Portas



Métodos de desarrollo



VS



Arquitectura de Software

En la industria tecnológica, la **arquitectura de software** es crucial para el desarrollo de productos eficientes y escalables. Este conjunto de principios y prácticas define la estructura y comportamiento de los sistemas. La correcta aplicación de estos fundamentos es esencial para el éxito en proyectos tecnológicos.



Principios de Diseño

Los **principios de diseño** en la arquitectura de software guían la toma de decisiones para crear sistemas robustos y flexibles. La separación de **preocupaciones**, la **modularidad** y la **escalabilidad** son fundamentales. Estos principios impactan directamente en la **calidad y mantenibilidad del software**.



Universidad de La Sabana | Facultad de Ingeniería


Edificio K

Patrones Arquitectónicos

Los patrones arquitectónicos son soluciones probadas para desafíos comunes en el diseño de sistemas. Patrones como MVC, microservicios y serverless ofrecen estructuras sólidas para aplicaciones modernas. Comprender y aplicar estos patrones es esencial para el desarrollo exitoso de software.

Tecnologías emergentes

La evolución constante de las tecnologías emergentes impacta en la arquitectura de software. La adopción de contenedores, tecnologías de nube y arquitecturas sin servidor está transformando la forma en que se diseñan y despliegan sistemas. Estar al tanto de estas tendencias es crucial para la competitividad en la industria tecnológica.



Desafíos y Consideraciones

La arquitectura de software enfrenta desafíos y consideraciones únicos en la industria tecnológica. La **seguridad**, el **rendimiento** y la **interoperabilidad** son aspectos críticos a tener en cuenta. La gestión de datos, la integración de sistemas y la adopción de estándares son también áreas de enfoque clave.





Modelado orientado a objetos

El Modelado Orientado a Objetos es una metodología de diseño de software que se basa en la interacción de **objetos** para representar sistemas.

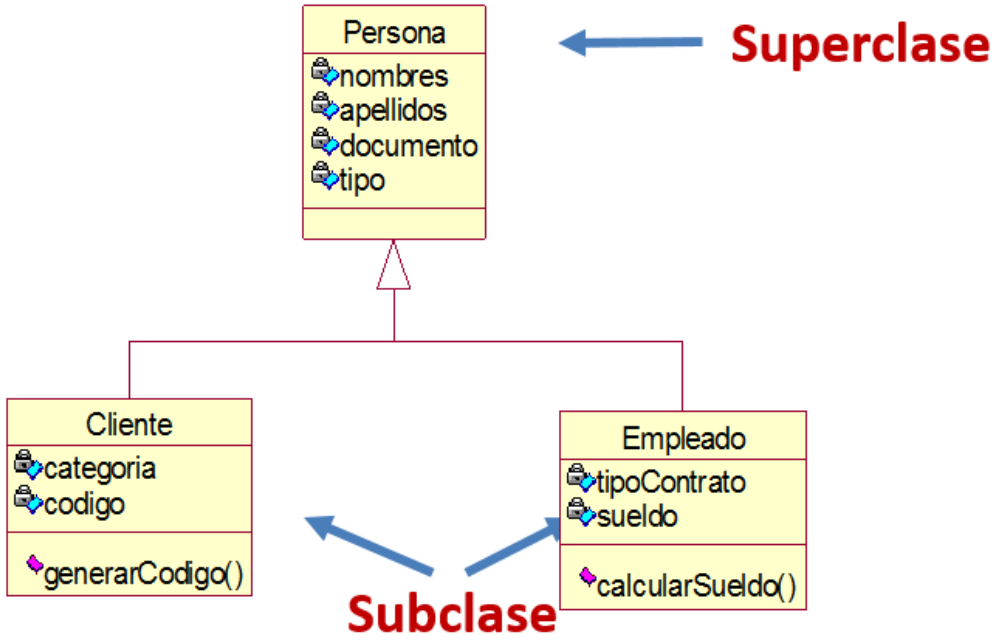
Utiliza conceptos como **clases**, **herencia** y **encapsulamiento** para organizar el código de manera eficiente.



Conceptos Básicos

En el Modelado Orientado a Objetos, un **objeto** es una instancia de una **clase** que posee **atributos** y **métodos**.

La **herencia** permite que las clases compartan características, mientras que el **encapsulamiento** protege la integridad de los datos.





Polimorfismo

El polimorfismo permite que un objeto pueda tomar diferentes formas, es decir, que un mismo método pueda comportarse de manera distinta según el objeto que lo invoque. Esto promueve la reutilización de código y la flexibilidad del sistema.

Universidad de La Sabana | Facultad de **Ingeniería**



Relaciones entre Objetos

En el Modelado Orientado a Objetos, los objetos pueden relacionarse entre sí mediante asociaciones, agregaciones y composiciones. Estas relaciones definen la interacción y la estructura del sistema.

Universidad de La Sabana | Facultad de **Ingeniería**


Edificio K

Modelamiento de software basado en UML

UML (Lenguaje de Modelado Unificado) es un estándar que proporciona una notación gráfica para visualizar, especificar, construir y documentar los artefactos de un sistema de software.

UML ha evolucionado a lo largo de los años desde su introducción en la década de 1990



Importancia

- **Comunicación Efectiva:**
  UML actúa como un lenguaje común entre desarrolladores, diseñadores, gerentes y otros stakeholders, facilitando la comunicación efectiva.
- **Visualización del Diseño:**
  Permite a los equipos visualizar la arquitectura y el diseño del sistema antes de la implementación, identificando posibles problemas y mejorando la calidad del software.
- **Documentación Clara:**
  Genera documentación visual clara y completa que sirve como referencia para el desarrollo, las pruebas y el mantenimiento.
- **Adaptabilidad y Flexibilidad:**
  Facilita la adaptación a cambios en los requisitos y la evolución continua del sistema a lo largo del tiempo.

Tipos de Diagramas

- Diagramas Estructurales
- Diagramas de Comportamiento
- Diagramas de Interacción



Diagrama de Clases

En UML es un tipo de diagrama de estructura estática que describe la estructura de un sistema mostrando las clases del sistema, sus atributos, operaciones (o métodos), y las relaciones entre los objetos.

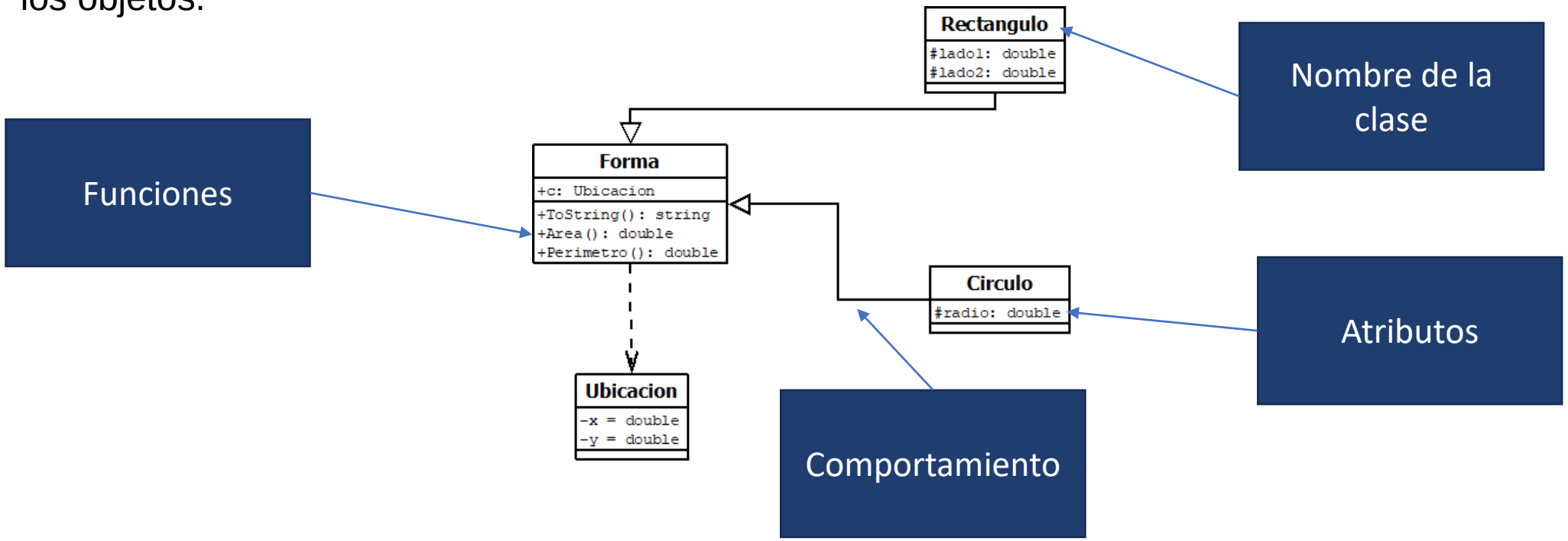




Diagrama de Objetos

En UML, diagrama que muestra una vista completa o parcial de los objetos de un sistema en un instante de ejecución específico.

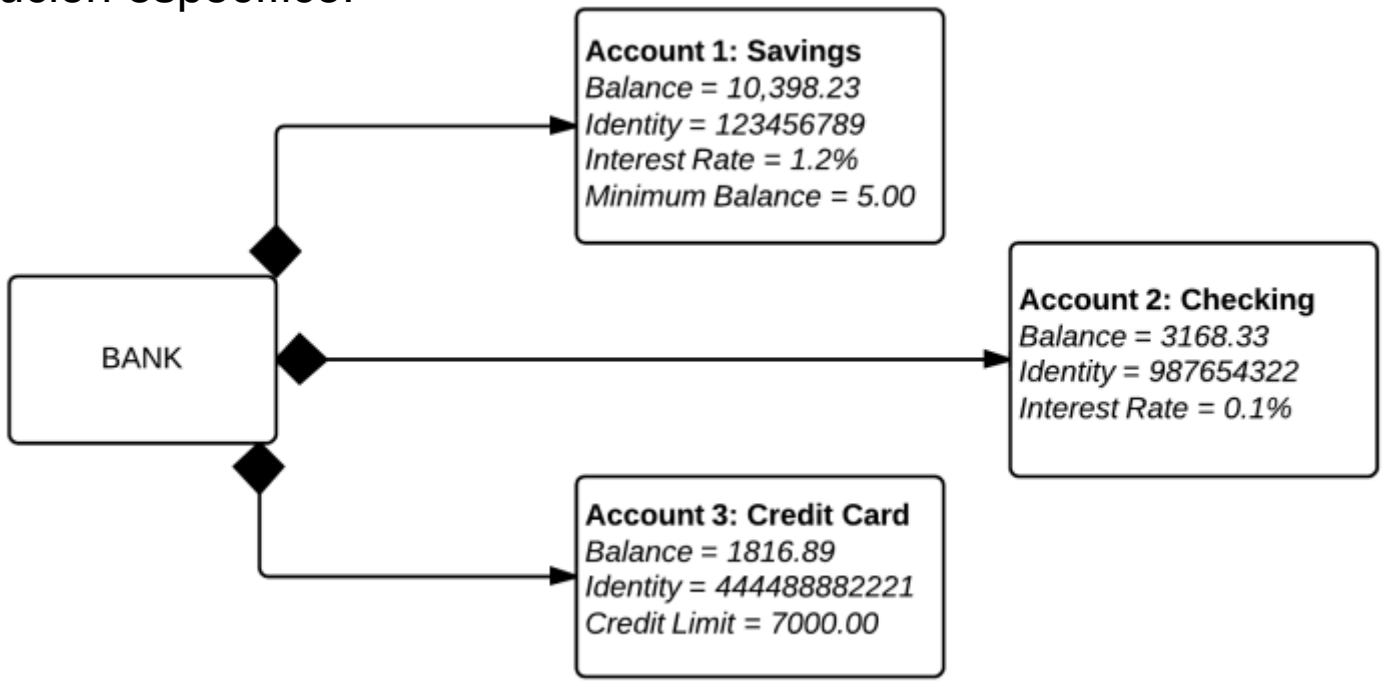


Diagrama de Secuencia

Enfoque en la visualización de interacciones entre objetos a lo largo del tiempo, destacando la secuencia de mensajes.

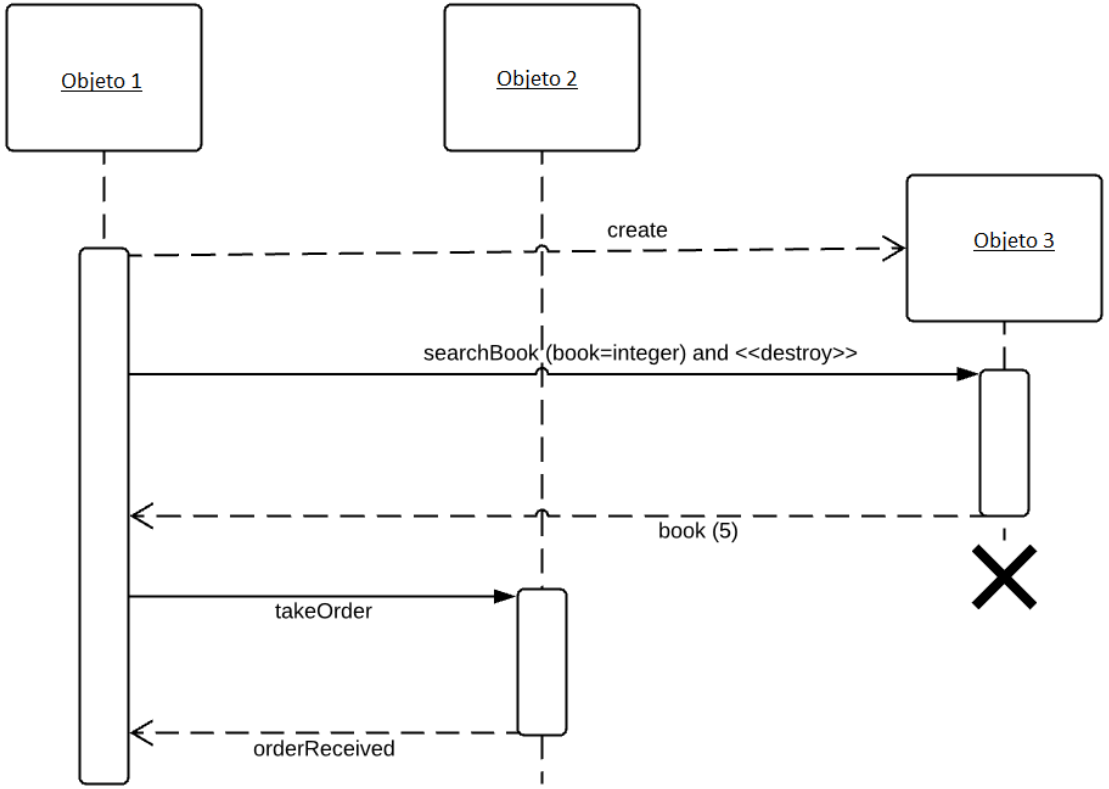






Diagrama de Actividad

Enfoque en la visualización de interacciones entre objetos a lo largo del tiempo, destacando lo que debe suceder en el sistema

- Demostrar la lógica de un algoritmo.
- Describir los pasos realizados en un caso de uso UML.
- Ilustrar un proceso de negocios o flujo de trabajo entre los usuarios y el sistema.
- Simplificar y mejorar cualquier proceso clarificando casos de uso complicados.
- Modelar elementos de arquitectura de software, tales como método, función y operación.

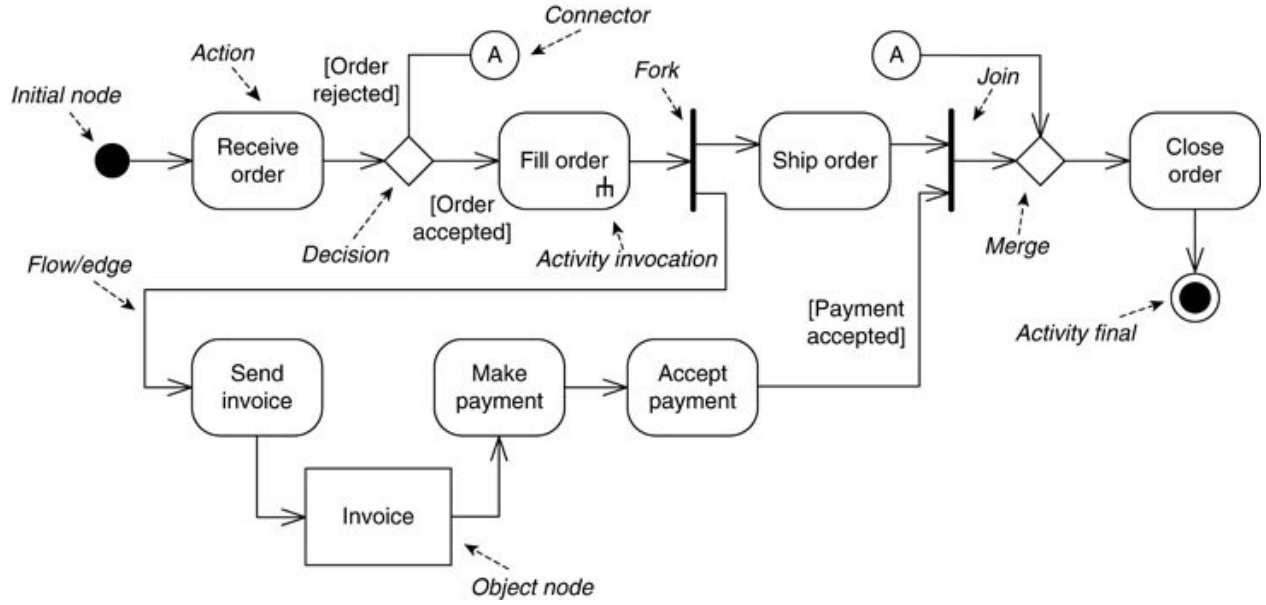


Diagrama de Estado

Explicación de cómo modelar estados y transiciones de un objeto, mostrando cómo un objeto responde a eventos específicos.

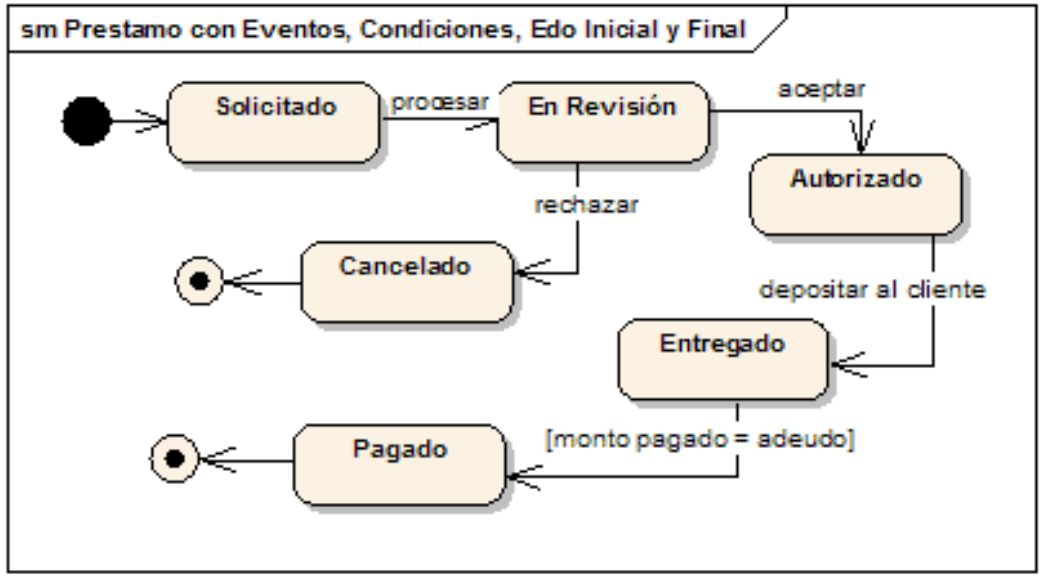






Diagrama de Casos de uso

Definición y representación gráfica de los casos de uso que describen las interacciones entre actores y el sistema.

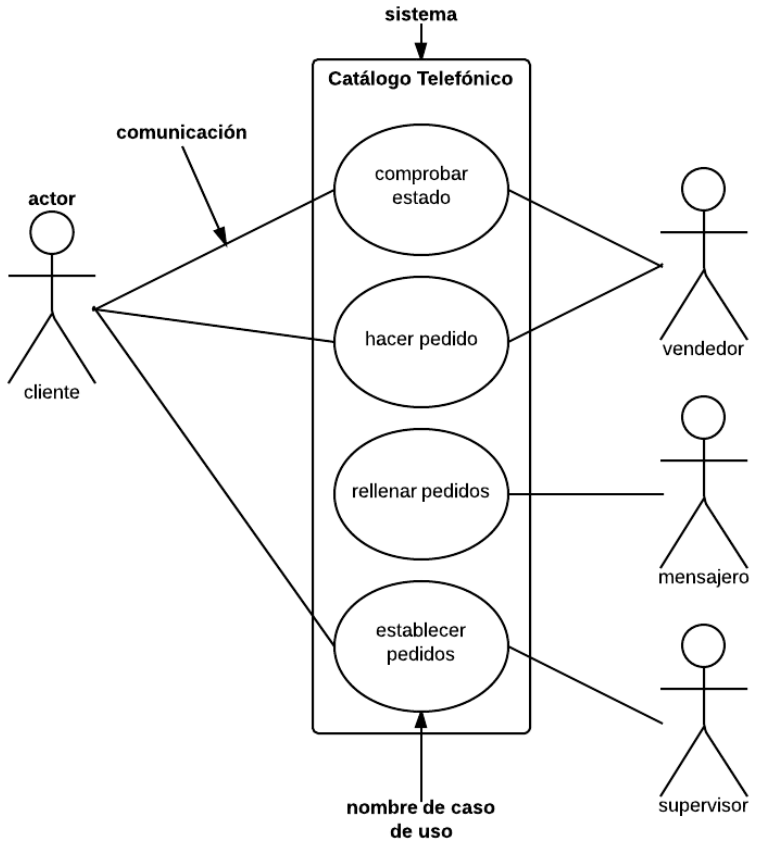
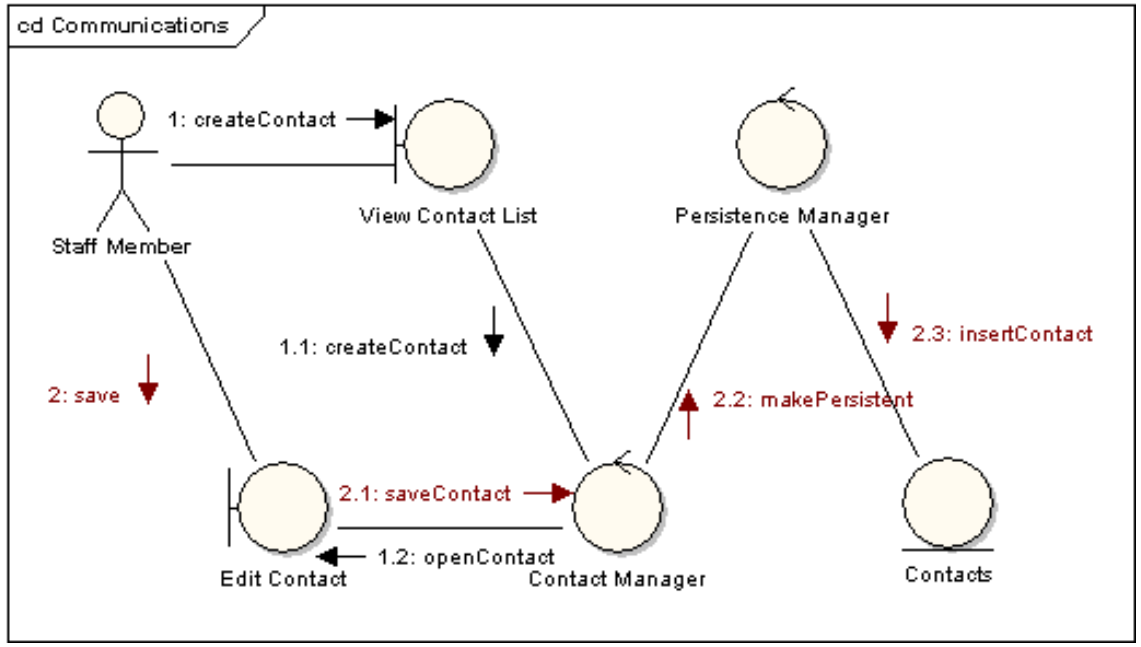


Diagrama de Comunicaciones

Enfoque en cómo representar las relaciones entre objetos y los mensajes que se intercambian.





¿Cuándo se usa?

La aplicación efectiva del Lenguaje de Modelado Unificado (UML) en el ciclo de desarrollo de software es crucial para garantizar una comunicación clara y una comprensión precisa de los diferentes aspectos del sistema en desarrollo.

UML se integra en varias etapas del ciclo de vida del desarrollo de software, cada una de las cuales contribuye a una comprensión más completa y una implementación exitosa del sistema.



Universidad de La Sabana | Facultad de Ingeniería


Zonas de estudio, Plaza San José María

Actividad

La concesionaria "Automóviles del Futuro" está buscando mejorar su sistema de gestión de autos mediante la implementación de Programación Orientada a Objetos (POO). La concesionaria desea que diseñes un sistema que permita gestionar la información de los autos disponibles en el inventario. Cada auto debe ser representado como un objeto con sus atributos y métodos específicos.

Universidad de La Sabana | Facultad de Ingeniería


Zonas de estudio, Plaza San José María

Actividad

Requerimientos:

1. **Clase Auto:**
   - Diseña una clase llamada **Auto** que represente un automóvil.
   - La clase debe tener atributos como modelo, marca, año de fabricación, precio, y si está disponible para la venta.
   - Debe incluir un método para obtener información detallada del auto.
2. **Inventario de la Concesionaria:**
   - Crea una clase llamada **Concesionaria** que sirva como contenedor para gestionar múltiples autos.
   - Implementa métodos para agregar un nuevo auto al inventario y para mostrar todos los autos disponibles.
3. **Venta de Autos:**
   - Agrega un método a la clase **Concesionaria** para marcar un auto como vendido.
   - Este método debe verificar si el auto está disponible antes de marcarlo como vendido.
4. **Gestión de Precios:**
   - Implementa un método en la clase **Auto** para actualizar el precio de un auto.
   - Asegúrate de que esta funcionalidad esté integrada en el sistema de gestión.

Atributos de calidad

Portabilidad

Mantenibilidad

Usabilidad

Confiabilidad

Seguridad

Eficiencia

Adaptabilidad



Lago con vista al Edificio Ad Portas



Portabilidad

1 **Adaptabilidad**

Capacidad de funcionar en diferentes plataformas y entornos

2 **Reusabilidad**

Posibilidad de reutilizar componentes en múltiples sistemas

3 **Interoperabilidad**

Facilidad para integrarse con otros sistemas y aplicaciones

Mantenibilidad

Modularidad

Diseño de componentes independientes y reutilizables

Documentación

Información clara y completa del software

Escalabilidad

Capacidad de crecer y soportar cargas mayores



Usabilidad

Facilidad de Uso

Interfaz intuitiva y de fácil aprendizaje

Accesibilidad

Compatibilidad con necesidades de usuarios diversos

Estética

Diseño visualmente atractivo y agradable

Eficiencia

Optimización de tareas y flujos de trabajo



Lago con vista al Edificio Ad Portas



Confiabilidad

1

Robustez

Capacidad de manejar situaciones imprevistas

2

Disponibilidad

Tiempos de actividad y respuesta adecuados

3

Tolerancia a Fallos

Recuperación y continuidad del servicio

Seguridad

Cifrado

Protección de la información sensible

Control de Acceso

Restringir el acceso no autorizado

Respuesta a Incidentes

Detección y mitigación de amenazas



Eficiencia

1

Rendimiento

Optimización de recursos y tiempos de respuesta

2

Escalabilidad

Capacidad de adaptarse a cambios de carga

3

Consumo de Recursos

Minimización del uso de memoria y CPU





Adaptabilidad

Entorno Cambiante	Capacidad de ajustarse a nuevos requisitos
Evolución Tecnológica	Posibilidad de actualizar y mejorar el software
Necesidades del Usuario	Flexibilidad para satisfacer demandas cambiantes

Facultad de
Ingeniería



Universidad de
La Sabana